Name : Gaurav Hinduja
Div : D15C
Batch : C
Roll No : 50

MLDL Experiment 01

Aim: Implement Linear and Logistic Regression on real-world datasets.

1. Dataset Source

- Dataset Name: House Prices – Advanced Regression Techniques
- Source: Kaggle Competition
https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques
- Platform: Kaggle

2. Dataset Description

This dataset contains detailed residential home data from Ames, Iowa, used to predict house sale prices. The dataset includes comprehensive property features describing structural, location-based, and quality-related aspects of houses.

It is widely used for regression modeling, feature engineering, and advanced predictive analytics.

- Training Set Size: 1460 samples (rows) × 81 attributes (columns)
- Test Set Size: 1459 samples (rows) × 80 attributes (columns)

Target Variable

- SalePrice (Continuous) – Final selling price of the house in dollars.

Key Features

1. OverallQual – Overall material and finish quality (scale 1–10)

2. GrLivArea – Above ground living area (square feet)
3. GarageCars – Size of garage in car capacity
4. TotalBsmtSF – Total basement area (square feet)
5. YearBuilt – Original construction year
6. TotRmsAbvGrd – Total rooms above ground (excluding bathrooms)
7. LotArea – Lot size in square feet
8. Neighborhood – Physical location within Ames city limits
9. KitchenQual – Kitchen quality rating
10. FullBath – Number of full bathrooms

The dataset contains both numerical and categorical features. Preprocessing such as:

- Handling missing values
- Encoding categorical variables
- Feature scaling (optional)

is required before applying regression algorithms.

3. Mathematical Formulation of the Algorithm

Multiple Linear Regression models the relationship between several independent variables and a continuous dependent variable.

The equation is:

SalePrice=β0+β1X1+β2X2+...+βnXn+ϵSalePrice = \beta_0 + \beta_1X_1 + \beta_2X_2 + ... + \beta_nX_n + \epsilonSalePrice=β0+β1X1+β2X2+...+βnXn+ϵ

Where:

- X1,X2,...,XnX_1, X_2, ..., X_nX1,X2,...,Xn = Input features
- β0\beta_0β0 = Intercept
- β1,β2,...,βn\beta_1, \beta_2, ..., \beta_nβ1,β2,...,βn = Regression coefficients
- ϵ\epsilonϵ = Error term

The model estimates coefficients that minimize the sum of squared errors between actual and predicted house prices.

4. Algorithm Limitations
   1. Linearity Assumption: Assumes a linear relationship between house features and sale price.
   2. Sensitive to Outliers: Extremely high-priced luxury houses can distort predictions.
   3. Multicollinearity: Highly correlated features (e.g., GarageArea & GarageCars) can reduce model stability.
   4. Overfitting Risk: Large number of features may cause overfitting without regularization.

5. Methodology / Workflow
   1. Load training dataset (train.csv).
   2. Perform exploratory data analysis (EDA).
   3. Handle missing values.
   4. Encode categorical variables using One-Hot Encoding or Label Encoding.
   5. Select important features.
   6. Split dataset into training and validation sets (80:20).
   7. Train Multiple Linear Regression model.
   8. Predict SalePrice.
   9. Evaluate model performance using MSE and R^2 score.

6. Performance Analysis

● Metric 1: Mean Squared Error (MSE)

○ Measures the average squared difference between actual and predicted house prices.

MSE=1n∑(yi−y^i)2MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2MSE=n1∑(yi −y^i)2

● Metric 2: R-Squared (R^2)

○ Indicates the percentage of variance in SalePrice explained by the model.

R2=1−SSresSStotR^2 = 1 - \frac{SS_{res}}{SS_{tot}}R2=1−SStotSSres

Higher R^2 and lower MSE indicate better model performance.

Note: In Kaggle competition, Root Mean Squared Log Error (RMSLE) is also used as the official evaluation metric.

7. Hyperparameter Tuning

Standard Linear Regression has no tunable hyperparameters.

To improve performance and prevent overfitting, regularization techniques can be applied:

Ridge Regression (L2 Regularization)

● Tuned Parameter: alpha

● Tested Values: [0.1, 1, 10]

Larger alpha → Stronger regularization

Helps reduce overfitting and improve generalization.

<u>Logistic Regression:</u>

<u>2.</u> <u>Dataset Source:</u>

- **<u>Dataset Name:</u> Diabetes prediction dataset**
- **<u>Source:</u>** <u>https://www.kaggle.com/datasets/iammustafatz/diabetesprediction-dataset</u>

- **<u>Repository Owner:</u> Mohammed Mustafa**

<u>3. Dataset Description:</u>

This dataset contains medical information of patients used for predicting the presence of heart disease. The data represents relationships between clinical features and the likelihood of heart disease, making it suitable for classification modeling and predictive analytics.

- **Size:** 303 samples (rows) × 14 attributes (columns)
- **Target Variable:**
  - target (Binary: 0 or 1) –

0 = No Heart Disease
1 = Presence of Heart Disease

2. Key Features

1. age – Age of the patient.
2. sex – Gender of the patient (0 = female, 1 = male).
3. cp – Chest pain type (4 categories).
4. trestbps – Resting blood pressure (mm Hg).
5. chol – Serum cholesterol level (mg/dl).
6. fbs – Fasting blood sugar (>120 mg/dl).
7. restecg – Resting electrocardiographic results.
8. thalach – Maximum heart rate achieved.
9. exang – Exercise-induced angina.
10. oldpeak – ST depression induced by exercise.

11. slope – Slope of peak exercise ST segment.
12. ca – Number of major vessels colored by fluoroscopy.
13. thal – Thalassemia type.

The dataset is clean and widely used for beginner and intermediate classification problems. Some categorical features may require encoding before applying machine learning algorithms.

3. Mathematical Formulation of the Algorithm

Since this is a **classification problem**, Logistic Regression is commonly used.

Logistic Regression models the probability that a patient has heart disease based on multiple input features.

The logistic function (sigmoid function) is:

P(y=1|x)=11+e−(β0+β1x1+β2x2+...+βnxn)P(y=1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1x_1 + \beta_2x_2 + ... + \beta_nx_n)}}P(y=1|x)=1+e−(β0+β1x1+β2x2+...+βnxn)1

Where:

- x1,x2,...,xnx_1, x_2, ..., x_nx1,x2,...,xn are input features
- β0\beta_0β0 is intercept
- β1,β2,...,βn\beta_1, \beta_2, ..., \beta_nβ1,β2,...,βn are coefficients
- Output is probability between 0 and 1

If probability ≥ 0.5 → Heart Disease Present
Else → No Heart Disease

4. Algorithm Limitations

1. **Linearity Assumption:** Assumes linear relationship between input features and log-odds of heart disease.
2. **Sensitive to Outliers:** Extreme values may affect model performance.

3. **Multicollinearity:** Highly correlated features (e.g., cholesterol & age) may affect coefficient interpretation.
4. **Binary Limitation:** Standard Logistic Regression handles binary classification only.

5. Methodology / Workflow

1. Load CSV dataset.
2. Select input features and target variable (target).
3. Handle missing values (if any).
4. Encode categorical variables (if required).
5. Split dataset into training and testing sets (80:20).
6. Train Logistic Regression model.
7. Predict heart disease presence.
8. Evaluate model performance.

6. Performance Analysis

- **Metric 1: Accuracy**
  - Percentage of correctly classified patients.
- **Metric 2: Confusion Matrix**
  - Shows True Positives, True Negatives, False Positives, False Negatives.
- **Metric 3: Precision & Recall**
  - Precision – How many predicted positives are actually positive.
  - Recall – How many actual positives are correctly predicted.
- **Metric 4: F1-Score**
  - Harmonic mean of Precision and Recall.

Higher Accuracy, Precision, Recall, and F1-score indicate better model performance.

7. Hyperparameter Tuning

Logistic Regression has tunable parameters:

- **C** – Inverse of regularization strength
- Tested Values: [0.01, 0.1, 1, 10]

Smaller C → Stronger Regularization
Larger C → Less Regularization

Regularization helps reduce overfitting and improves generalization.

Code And Output:

Linear Regression

```
# ====================================================
# FINAL EXPERIMENT
# LINEAR + LOGISTIC REGRESSION
# ====================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn import metrics
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

print("Files in Directory:")
print(os.listdir())

# ====================================================
```

```
# PART 1 : LINEAR REGRESSION (Housing Dataset)
# ====================================================

print("\n\n========== LINEAR REGRESSION ==========")

# Load Housing Dataset
df1 = pd.read_csv("Housing.csv")

print("\n===== Dataset Preview =====")
print(df1.head())

# Convert categorical variables
df1 = pd.get_dummies(df1, drop_first=True)

print("\n===== Processed Dataset =====")
print(df1.head())

# Features and Target
X1 = df1.drop("price", axis=1)
y1 = df1["price"]

print("\n===== Features Used =====")
print(X1.columns)

# Train-Test Split
X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)

# Train Model
lin_model = LinearRegression()
lin_model.fit(X1_train, y1_train)

# Prediction
y1_pred = lin_model.predict(X1_test)

# Evaluation
mae = metrics.mean_absolute_error(y1_test, y1_pred)
mse = metrics.mean_squared_error(y1_test, y1_pred)
rmse = np.sqrt(mse)
r2 = metrics.r2_score(y1_test, y1_pred)
```

```
print("\n===== Model Performance =====")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R-squared Score (R2): {r2:.4f} ({r2*100:.2f}%)")

print("\n===== Regression Equation Details =====")
print(f"Intercept (b0): {lin_model.intercept_:.4f}")

coeff_df = pd.DataFrame(lin_model.coef_, X1.columns, columns=["Coefficient"])
print(coeff_df)

# Plot
plt.figure(figsize=(8,6))
plt.scatter(y1_test, y1_pred, alpha=0.7)
plt.plot([y1.min(), y1.max()], [y1.min(), y1.max()])
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Housing Prices")
plt.grid(True)
plt.show()
# ====================================================
# LOGISTIC REGRESSION WITH HYPERPARAMETER TUNING
# ====================================================

print("\n\n========== LOGISTIC REGRESSION ==========")

df2 = pd.read_csv("heart_disease_uci.csv")

# Convert categorical columns
df2 = pd.get_dummies(df2, drop_first=True)

# Handle missing values
df2 = df2.fillna(df2.median())

# Features and target
X2 = df2.drop("num", axis=1)
y2 = df2["num"]

# Convert to binary
```

```
y2 = y2.apply(lambda x: 1 if x > 0 else 0)

# Train-Test Split
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X2_train = scaler.fit_transform(X2_train)
X2_test = scaler.transform(X2_test)

# ----------------------------------------------------
# Hyperparameter Tuning (C values)
# ----------------------------------------------------

print("\n===== Hyperparameter Tuning =====")

C_values = [0.01, 0.1, 1, 10, 100]
best_acc = 0
best_C = None

for c in C_values:
    model = LogisticRegression(C=c, max_iter=10000)
    model.fit(X2_train, y2_train)
    y_pred_temp = model.predict(X2_test)
    acc_temp = accuracy_score(y2_test, y_pred_temp)

    print(f"C = {c} --> Accuracy = {acc_temp:.4f}")

    if acc_temp > best_acc:
        best_acc = acc_temp
        best_C = c

print(f"\nBest C Value: {best_C}")
print(f"Best Accuracy: {best_acc:.4f}")

# ----------------------------------------------------
# Train Final Model with Best C
# ----------------------------------------------------
```

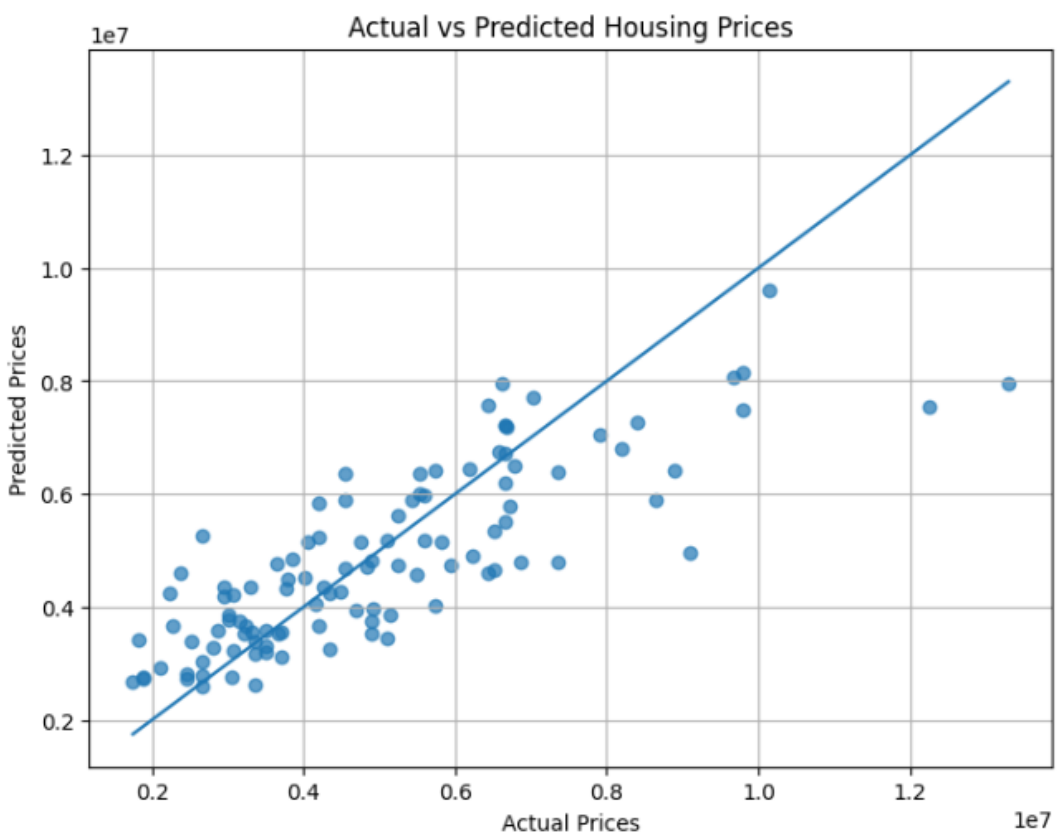
```
log_model = LogisticRegression(C=best_C, max_iter=10000)
log_model.fit(X2_train, y2_train)

y2_pred = log_model.predict(X2_test)

# Metrics
acc = accuracy_score(y2_test, y2_pred)
prec = precision_score(y2_test, y2_pred)
rec = recall_score(y2_test, y2_pred)
f1 = f1_score(y2_test, y2_pred)

print("\n===== Final Model Performance =====")
print(f"Accuracy: {acc:.4f} ({acc*100:.2f}%)")
print(f"Precision: {prec:.4f} ({prec*100:.2f}%)")
print(f"Recall: {rec:.4f} ({rec*100:.2f}%)")
print(f"F1 Score: {f1:.4f} ({f1*100:.2f}%)")

# Confusion Matrix
cm = confusion_matrix(y2_test, y2_pred)
tn, fp, fn, tp = cm.ravel()

print("\n===== Confusion Matrix Values =====")
print(f"TP: {tp}, TN: {tn}, FP: {fp}, FN: {fn}")

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()

# ----------------------------------------------------
# Sigmoid Curve
# ----------------------------------------------------

z = log_model.decision_function(X2_test)
prob = log_model.predict_proba(X2_test)[:,1]

idx = np.argsort(z)

plt.figure(figsize=(8,6))
```

```
plt.scatter(z, prob, alpha=0.3)
plt.plot(z[idx], prob[idx])
plt.axhline(0.5, color='red')
plt.xlabel("Linear Decision Function (z)")
plt.ylabel("Probability")
plt.title("Logistic Regression Sigmoid Curve")
plt.grid(True)
plt.show()
```

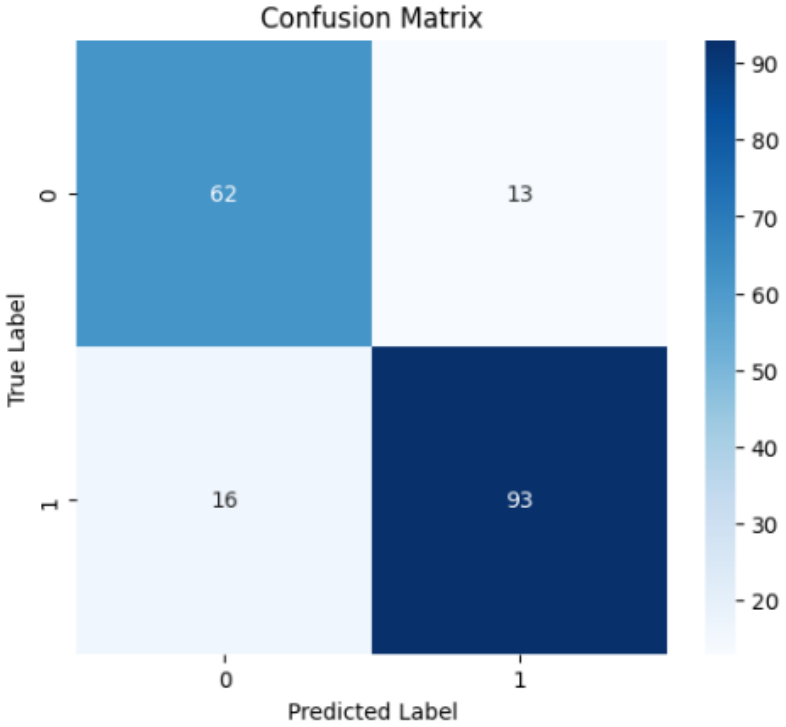
Output :

```
========== LINEAR REGRESSION ==========

===== Dataset Preview =====
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished

===== Processed Dataset =====
      price  area  bedrooms  bathrooms  stories  parking  mainroad_yes  \
0  13300000  7420         4          2        3        2          True
1  12250000  8960         4          4        4        3          True
2  12250000  9960         3          2        2        2          True
3  12215000  7500         4          2        2        3          True
4  11410000  7420         4          1        2        2          True

   guestroom_yes  basement_yes  hotwaterheating_yes  airconditioning_yes  \
0          False         False                False                 True
1          False         False                False                 True
2          False          True                False                False
3          False          True                False                 True
4           True          True                False                 True

   prefarea_yes  furnishingstatus_semi-furnished  furnishingstatus_unfurnished
0          True                            False                         False
1         False                            False                         False
2          True                             True                         False
3          True                            False                         False
4         False                            False                         False

===== Features Used =====
Index(['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'mainroad_yes',
       'guestroom_yes', 'basement_yes', 'hotwaterheating_yes',
       'airconditioning_yes', 'prefarea_yes',
       'furnishingstatus_semi-furnished', 'furnishingstatus_unfurnished'],
      dtype='object')
```

```
===== Model Performance =====
Mean Absolute Error (MAE): 970043.4039
Mean Squared Error (MSE): 1754318687330.6638
Root Mean Squared Error (RMSE): 1324506.9601
R-squared Score (R2): 0.6529 (65.29%)

===== Regression Equation Details =====
Intercept (b0): 260032.3576
                                  Coefficient
area                             2.359688e+02
bedrooms                         7.677870e+04
bathrooms                        1.094445e+06
stories                          4.074766e+05
parking                          2.248419e+05
mainroad_yes                     3.679199e+05
guestroom_yes                    2.316100e+05
basement_yes                     3.902512e+05
hotwaterheating_yes              6.846499e+05
airconditioning_yes              7.914267e+05
prefarea_yes                     6.298906e+05
furnishingstatus_semi-furnished -1.268818e+05
furnishingstatus_unfurnished    -4.136451e+05
```



```
========== LOGISTIC REGRESSION ==========

===== Hyperparameter Tuning =====
C = 0.01 --> Accuracy = 0.8315
C = 0.1 --> Accuracy = 0.8315
C = 1 --> Accuracy = 0.8424
C = 10 --> Accuracy = 0.8370
C = 100 --> Accuracy = 0.8315

Best C Value: 1
Best Accuracy: 0.8424

===== Final Model Performance =====
Accuracy: 0.8424 (84.24%)
Precision: 0.8774 (87.74%)
Recall: 0.8532 (85.32%)
F1 Score: 0.8651 (86.51%)

===== Confusion Matrix Values =====
TP: 93, TN: 62, FP: 13, FN: 16
```



Conclusion:

This experiment demonstrated the practical implementation of Linear Regression for medical insurance charge prediction and Logistic Regression for diabetes prediction using Kaggle datasets. Both models successfully captured relationships between input features and their respective target variables, showing the effectiveness of supervised learning techniques for solving real-world problems.

The experiment highlighted the importance of data preprocessing, feature selection, feature scaling, and train–test splitting in building reliable machine learning models. Performance evaluation using metrics such as Mean Squared Error, R^2 score, Accuracy, and Confusion Matrix provided insights into model effectiveness. Overall, this experiment strengthened understanding of regression and classification algorithms and their application in predictive analytics in healthcare and insurance domains.